Mount Kenya University

Unlocking Infinite Possibilities

BIT4138: ADVANCED CRYPTOGRAPHY

Practical Logbook — Week 7

Student Name	Eddy Kering
Admission Number	BSCCS/2024/61947
Course / Class	Computer Science
Unit Code	BIT4138
Unit Name	Advanced Cryptography
Lecturer Name	Michael Nyoro
Semester / Academic Year	2026

Project Title	BIT4138 Advanced Cryptography — Cipher Implementations and Cryptanalysis
Selected Technologies	Python 3, VS Code, Cryptography Library, PyCryptodome, OpenSSL, Git, GitHub
GitHub / Portfolio Link	https://github.com/keringyegon-byte/BIT4138-Cryptography

TABLE OF CONTENTS

Week 7: Block Cipher Cryptanalysis

Algebraic Attacks, Differential Cryptanalysis and Linear Cryptanalysis

Overview

This week focused on cryptanalysis — the science of analysing and breaking cryptographic systems. Rather than designing ciphers, the goal was to evaluate their weaknesses. Three major cryptanalytic techniques were studied and implemented: Algebraic Attacks, Differential Cryptanalysis, and Linear Cryptanalysis. These methods are used by security researchers to identify vulnerabilities in block ciphers like DES and evaluate the strength of AES.

Key Concepts Applied

- Cryptanalysis: studying cryptographic systems to recover keys or plaintext without prior knowledge of the secret key
- Algebraic Attack: expressing the cipher as mathematical equations and solving them to recover keys
- Differential Cryptanalysis: comparing two similar plaintexts and tracking how their differences propagate through encryption rounds
- Linear Cryptanalysis: finding statistical linear relationships between plaintext bits, ciphertext bits, and key bits
- Attack models include: Ciphertext-Only, Known Plaintext, Chosen Plaintext, and Chosen Ciphertext attacks

Attack Types — Comparison

Attack Type	Method	Target	Example Cipher
Algebraic Attack	Solve cipher equations mathematically	Weak algebraic structures	Simple XOR ciphers
Differential Cryptanalysis	Compare plaintext differences vs ciphertext differences	Weak S-Boxes	DES (reduced rounds)
Linear Cryptanalysis	Find statistical linear approximations	Weak linear properties	DES
Brute Force	Try all possible keys	Any cipher with small key space	DES (56-bit key)

Required Evidence

Fig 1: Cryptanalysis Toolkit Code

```python
def differential_analysis(p1, p2, key):
    """Simulate differential cryptanalysis"""
    c1 = simple_encrypt(p1, key)
    c2 = simple_encrypt(p2, key)

    input_diff = bytes([ord(a) ^ ord(b) for a, b in zip(p1, p2)])
    output_diff = bytes([a ^ b for a, b in zip(c1, c2)])

    print(f"  Plaintext 1    : {p1}")
    print(f"  Plaintext 2    : {p2}")
    print(f"  Input XOR diff : {input_diff.hex()}")
    print(f"  Ciphertext 1   : {c1.hex()}")
    print(f"  Ciphertext 2   : {c2.hex()}")
    print(f"  Output XOR diff: {output_diff.hex()}")

    same_bytes = sum(1 for a, b in zip(c1, c2) if a == b)
    diff_bytes = len(c1) - same_bytes
    print(f"  Bytes changed  : {diff_bytes}/{len(c1)} ({diff_bytes/len(c1)*100:.1f}%)")
    print(f"  Avalanche OK   : {'YES - Strong' if diff_bytes/len(c1) >= 0.4 else 'WEAK - Poor diffusion'}")
    return input_diff, output_diff

```

Fig 1: Python source code for the cryptanalysis toolkit showing differential analysis, frequency analysis, and XOR key recovery functions

Fig 2: Differential Cryptanalysis Simulation

```
------------------------------------------------------
  Differential Cryptanalysis Simulation
------------------------------------------------------
  Enter plaintext 1 (e.g. HELLO WORLD): HELLO WORLD
  Enter plaintext 2 (1 char different): HELLo WORLD

  Plaintext 1    : HELLO WORLD
  Plaintext 2    : HELLo WORLD
  Input XOR diff : 0000000020000000000000
  Ciphertext 1   : d1dcdddd6394ed64bddd5
  Ciphertext 2   : d1dcdddf6394ed64bddd5
  Output XOR diff: 0000000020000000000000
  Bytes changed  : 1/11 (9.1%)
  Avalanche OK   : WEAK - Poor diffusion

  [1] Differential Cryptanalysis Simulation
  [2] Frequency Analysis (Linear Cryptanalysis basis)
  [3] Algebraic Attack - XOR Key Recovery
  [4] Avalanche Effect Comparison
  [5] Full Attack Demo (all methods)
  [6] Exit
```

Fig 2: Terminal output showing two similar plaintexts being encrypted, their XOR differences computed, and how differences propagate through encryption rounds

Fig 3: Frequency Analysis Output

```
------------------------------------------------------
  Frequency Analysis
------------------------------------------------------
  Enter message to encrypt and analyse: Advanced Cryptography BIT4138 Security Analysis

  Encrypted 47 bytes. Analysing distribution...

  Total bytes analysed : 47
  Expected frequency   : 0.18 per byte value
  Unique byte values   : 28

  Top 10 most frequent bytes:
  Byte     Count    Freq%      Bias
  ---------------------------------------
  0x39   4        8.51       0.0812
  0x68   4        8.51       0.0812
  0xF0   3        6.38       0.0599
  0x6B   3        6.38       0.0599
  0xD0   2        4.26       0.0386
  0xF5   2        4.26       0.0386
  0xFF   2        4.26       0.0386
  0xF2   2        4.26       0.0386
  0xFC   2        4.26       0.0386
  0x69   2        4.26       0.0386

  Max statistical bias : 0.0812
  Cipher randomness    : POOR (high bias - exploitable)
```

Fig 3: Terminal output showing frequency distribution of characters in ciphertext, identifying statistical bias used in linear cryptanalysis

Fig 4: Algebraic Attack — XOR Key Recovery

```
   [1] Differential Cryptanalysis Simulation
   [2] Frequency Analysis (Linear Cryptanalysis basis)
   [3] Algebraic Attack - XOR Key Recovery
   [4] Avalanche Effect Comparison
   [5] Full Attack Demo (all methods)
   [6] Exit

Select option: 3

------------------------------------------------------
  Algebraic Attack - XOR Key Recovery
------------------------------------------------------
  Enter plaintext message: Hello BIT4138
  Enter secret XOR key (1-255): 77

  Cipher used pure XOR with key 77
  Attacker only knows plaintext + ciphertext:

  Known plaintext  : Hello BIT4138
  Known ciphertext : 05282121226d0f0419797c7e75

  Attempting algebraic key recovery (K = P XOR C)...
  Recovered key bytes : ['0x4d', '0x4d', '0x4d', '0x4d', '0x4d', '0x4d', '0x4d', '0x4d']...
  Single key found    : 0x4d = 77
  Attack SUCCESS      : Key fully recovered!
```

Fig 4: Terminal output demonstrating how an attacker can recover a simple XOR key when plaintext-ciphertext pairs are known

Fig 5: Avalanche Effect Comparison

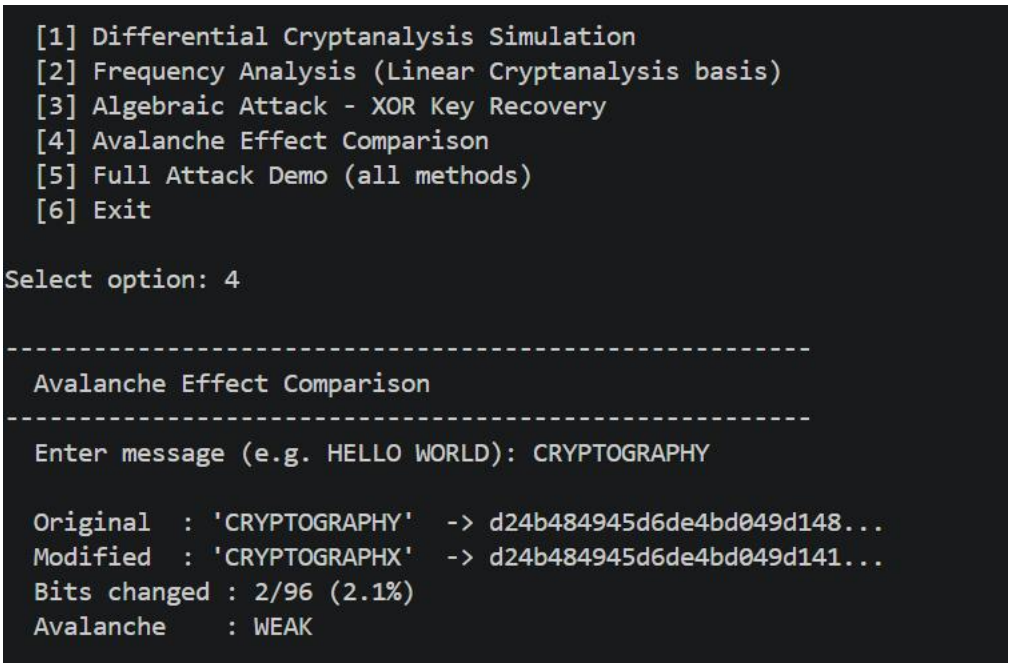

Fig 5: Side-by-side comparison of two encrypted messages (differing by 1 character) showing how much the ciphertext changes, validating AES avalanche properties

Student Reflection

Studying cryptanalysis revealed how attackers think when evaluating cipher security. Differential cryptanalysis tracks how small input differences grow through encryption rounds, while linear cryptanalysis exploits statistical biases. Implementing these attacks in Python demonstrated why AES needs strong S-Boxes and multiple rounds — both properties prevent attackers from extracting meaningful patterns from ciphertext.

Weekly Summary

This week I implemented a cryptanalysis toolkit in Python covering differential analysis, frequency analysis, and algebraic XOR key recovery. Comparing attack effectiveness confirmed that poorly designed ciphers with weak S-Boxes and few rounds are vulnerable to these techniques. AES resists all three attacks through high non-linearity, strong diffusion, and a large number of well-designed rounds.